

STD-008 断言规范

制定日：2023-04-01 / 最终修订：2025-08-10 / 管理：QA 基盘团队

一条没有断言的案例，跑通了也证明不了任何事情 ——

它只说明那几个点击没有报错。本篇规定断言的最低要求、粒度和常见反模式。

STD-008 至少一个断言 (ERROR)

规定

每条案例**必须包含至少一个 `ASSERT\_\*` 步骤**。不满足的案例无法保存为 `ACTIVE` 状态。

`ASSERT\_\*` 指以下三个 action：

- `ASSERT\_TEXT`
- `ASSERT\_VISIBLE`
- `ASSERT\_NOT\_EXIST`

为什么这是 ERROR 级

没有断言的案例会产生**最坏的一种结果：稳定地给出错误的安全感**。

它每次都通过，出现在通过率统计里，让人以为这块功能被覆盖了。而实际上被测系统即使返回了完全错误的结果，只要页面没崩、元素还在，案例照样通过。

这比没有案例更糟。没有案例时人们知道这块没被覆盖，会用别的方式验证；有一条假案例时，人们会停止验证。

常见的漏写场景

...

只有操作没有验证：

1. OPEN_URL 商品列表页
 2. INPUT 搜索框 "ノートPC"
 3. CLICK 搜索按钮
- (结束)

...

这条案例只能发现「搜索按钮点不动」这一种问题。搜出来的结果对不对、有没有结果、报没报错，一概不知。

正确的写法至少要加一步：

...

```
4. ASSERT_VISIBLE //div[@data-testid="search-result-list"]
```

...

更好的写法是断言具体内容：

...

```
4. ASSERT_TEXT //div[@data-testid="result-count"] expected: 件
```

```
5. ASSERT_TEXT //li[@data-testid="result-item"][1] expected: ノートPC
```

...

断言粒度

断言什么

优先断言**业务上有意义的结果**，而不是技术性的中间状态。

| 优先级 | 断言对象 | 示例 |

| --- | --- | --- |

| 高 | 业务结果数据 | 订单金额、库存数量、状态文案 |

| 中 | 明确的反馈信息 | 「保存成功」提示、错误消息文本 |

| 低 | 页面结构存在性 | 某个 div 可见 |

`ASSERT\_VISIBLE` 一个容器元素是最弱的断言 —— 容器在，不代表里面的内容对。能用

`ASSERT\_TEXT` 断言具体值的地方就不要只断言可见。

关键操作后要断言

不要把所有断言都堆在案例末尾。

...

断言全部在最后：

1-8. 一系列操作

```
9. ASSERT_TEXT 订单状态 = 已支付
```

...

如果第 4 步实际没生效，失败会在第 9 步暴露，截图显示的是第 9 步那一刻的页面 ——

报错位置离真正的问题有 5 步之远。

...

关键节点各有断言：

1-3. 填写收货信息

```
4. ASSERT_VISIBLE //div[@data-testid="address-confirmed"]
5-7. 选择支付方式并提交
8. ASSERT_TEXT //span[@data-testid="order-status"] expected: 支払い済み
...
```

这样第 4 步失败就在第 4 步报出来。****让失败尽早暴露在它真正发生的地方****，是缩短排查时间最有效的手段。

避免过度断言

断言不是越多越好

与上一节并不矛盾：关键节点要有断言，但****不要把页面上每个元素都断言一遍****。

过度断言的代价：

1. ****维护成本**** —— 页面改一次文案，一批案例要改
2. ****失败噪音**** —— 一个无关的 UI 调整导致大量案例变红，真正的问题被淹没
3. ****测试意图模糊**** —— 断言了 20 个东西的案例，失败时不知道到底哪个才是这条案例真正关心的

判断标准

对每个断言问一句：****这条断言失败时，是不是意味着这条案例想测的东西坏了？****

答案是「不一定」的断言就该删掉。

...

案例意图：验证下单流程能正常完成

```
ASSERT_TEXT 订单状态 = 支払い済み    直接对应测试意图
ASSERT_TEXT 订单金额 = ¥12,800      金额算错就是这条案例该发现的
ASSERT_VISIBLE 页面右上角的客服图标  和下单流程无关，改版就会误报
ASSERT_TEXT 页脚版权文字            完全无关
...
```

文案断言要谨慎

断言 UI 文案是维护成本最高的一类断言，因为文案改动频繁且往往不通知测试。

建议：

- 断言****部分匹配****而非完整句子（默认就是包含匹配，不要随便加 `^...\$`）
- 优先断言****不会变的关键词****（金额、编号、状态值）而不是完整提示语
- 多语言站点尤其注意 —— 断言了中文文案的案例，在日文环境下必然失败

反模式

断言自己刚输入的值

```

3. INPUT 用户名输入框 "test\_user"
4. ASSERT\_TEXT 用户名输入框 expected: test\_user

```

这只能验证了浏览器把字符填进了输入框，没有验证被测系统的任何行为。除非在排查「输入被自动转换成全角」这类具体问题，否则没有意义。

用 ASSERT_NOT_EXIST 做正向验证

```

```
ASSERT_NOT_EXIST //div[@class="error-message"]
```

```

「没有错误提示」不等于「操作成功」。页面可能还没加载完，可能跳到了别的页面，也可能成功提示和错误提示用的是同一个容器。

****否定断言只能用于验证「确实应该消失 / 不该出现」的东西****，不能用来替代正向验证。而且使用前必须先有一个正向断言确认页面已就绪——详见《待機戰略規約》STD-006 一节。

断言与业务无关的技术细节

```

```
ASSERT_VISIBLE //div[@class="ant-spin-container"]
```

```

断言了组件库的内部结构。UI 组件库升级一次，这类断言全部失效，而它们本来也没验证任何业务逻辑。

评审要点

以下项目无法自动检查，由评审人确认（见《案例评审 checklist》）：

- [] 断言对应的是业务结果，不是中间状态
- [] 关键操作后有断言，失败能就近暴露
- [] 没有与测试意图无关的断言
- [] 否定断言前有正向断言
- [] 文案断言用了部分匹配，且断言的是稳定关键词